Scribble to Digital

Team ID : SWTID-2026-4037

Team Leader : Mithun R

Team member : Manikandan L

Team member : Nithish kumar S

Team member : Bharath kumar E

ABSTRACT

Handwritten notes are commonly used by students and professionals for learning, meetings, and daily task management. However, converting handwritten notes into digital format is often difficult due to unclear handwriting, poor lighting conditions, and mixed content such as notes and task lists. This process usually requires manual rewriting, which is time-consuming and inefficient.

The Scribble to Digital system is an AI-powered application designed to convert handwritten notes into clean and structured digital text. The system first enhances the quality of uploaded note images to improve readability. It then uses Optical Character Recognition (OCR) to extract text from the handwritten content. After extraction, Generative AI is applied to correct grammar, improve sentence structure, and automatically identify important To-Do tasks from the notes.

The application is developed using Python as the backend processing engine and Streamlit as the frontend interface. This system helps users save time, improve productivity, and organize handwritten information more efficiently by transforming messy handwritten notes into clear digital content.

INTRODUCTION

In today's digital world, students and professionals still rely heavily on handwritten notes for learning, meetings, and daily planning. However, handwritten notes can often be messy, unclear, or difficult to organize.

Converting handwritten notes into digital format manually is time-consuming and inefficient. With the advancement of Artificial Intelligence, technologies like OCR and Generative AI can help automate this process.

The Scribble to Digital system uses AI techniques to convert handwritten notes into structured digital text. It enhances uploaded images, extracts the text, corrects grammar, and organizes important tasks automatically. The application is built using Python for backend processing and Streamlit for the frontend interface, making it easy for users to upload and process their notes.

PROBLEM STATEMENT

Students and professionals often take handwritten notes for studying, meetings, and daily tasks. However, these notes are frequently difficult to read due to poor handwriting, bad lighting, or messy formatting. Additionally, handwritten notes often contain mixed information such as explanations, reminders, and tasks.

Manually converting these notes into digital format is time-consuming and inefficient. Therefore, there is a need for an intelligent system that can automatically convert handwritten notes into clear digital text while identifying and organizing important tasks.

Project Workflow

1. Gemini AI Setup & Initialization

Activity 1.1: Set up Google AI Studio with Gemini API

- Create or sign in to Google AI Studio account
- Generate API key for Gemini model access
- Enable Gemini 2.5 Flash model
- Test API integration using a sample prompt

Activity 1.2: Configure API Keys and Environment

- Store GEMINI_API_KEY securely in .env file
- Load environment variables using python-dotenv
- Configure model parameters:
    - Temperature
    - Max output tokens
    - Response format

- Validate API connectivity inside Streamlit app

Activity 1.3: Design AI Prompts for Note Processing

- Create prompt template for:
    - Grammar correction
    - Sentence reconstruction
    - Formatting messy text into structured paragraphs
- Design separate prompt for:
    - To-do task extraction
    - Task prioritization (if needed)
- Test prompt effectiveness with sample handwritten text.

2. Core Functionality Development

Activity 2.2: Implement Image Processing Module

- Accept image upload (JPG/PNG)
- Apply preprocessing using OpenCV:
    - Convert to grayscale
    - Adjust brightness & contrast
    - Remove noise
    - Apply thresholding
- Display enhanced image preview in Streamlit

Activity 2.3: Implement OCR Extraction

- Integrate Tesseract or EasyOCR
- Extract raw handwritten text
- Clean unnecessary symbols
- Display raw OCR output for comparison

Activity 2.4: AI-Based Contextual Processing

- Send OCR text to Gemini API
- Perform:
    - Grammar correction
    - Sentence completion
    - Context-based word guessing
    - Formatting into structured notes
- Extract action items separately
- Return:
    - Clean digital text
    - Structured to-do list

3. Streamlit UI Implementation

Activity 3.1: Build Interactive UI Components

- st.file_uploader() for image upload
- st.image() for displaying uploaded & enhanced image
- st.button() for processing steps
- st.spinner() for loading indication
- st.text_area() for displaying extracted text

Activity 3.2: Display Structured Results

- Section 1: Enhanced Image
- Section 2: Raw OCR Text
- Section 3: AI-Corrected Digital Notes
- Section 4: Extracted To-Do Tasks (bullet format)

Optional:

- Checkbox to show/hide raw OCR
- Download buttons for final output

Activity 3.3: Add Export Functionality

- Generate downloadable:
    - PDF (using ReportLab)
    - Word file (using python-docx)
- Use st.download_button()
- Allow preview before download

4. Testing & Optimization

Activity 4.1: Functional Testing

- Test with:
    - Clear handwriting
    - Messy handwriting
    - Mixed text + tasks
    - Bullet points
- Validate OCR accuracy
- Validate AI corrections

Activity 4.2: Evaluate AI Response Quality

- Check:
    - Grammar correction accuracy
    - Logical sentence flow
    - Proper formatting
    - Correct task extraction
- Ensure no hallucinated content

Activity 4.3: Performance Optimization

- Optimize image preprocessing steps

- Reduce API token usage
- Add error handling for:
    - Empty image
    - API failure
    - OCR failure
- Improve prompt clarity

Activity 4.4: Ensure Export Quality

- Validate PDF formatting
- Check Word document structure
- Ensure file compatibility
- Test downloads across browsers

Activity 1.2: Configure Environment and Test Connection (Streamlit Setup)

1: Create. env File

```
scribble_to_digital/
    app.py
    .env
```

2: Install Required Dependencies

```
pip install streamlit google-generativeai python-dotenv pillow opencv-python pytesseract
```

3: Test Gemini API Connection in Streamlit

```python
import streamlit as st
import google.generativeai as genai
import os
from dotenv import load_dotenv

# Load API key
load_dotenv()
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

# Select model
model = genai.GenerativeModel("models/gemini-2.5-flash")

st.title("Gemini API Test - Scribble to Digital")

if st.button("Test AI Connection"):
    response = model.generate_content(
        "Correct this sentence: i am go school tomorrow"
    )
    st.write("AI Response:")
```

Activity 1.3: Validate Connectivity and Response Quality

1: Grammar Correction

```
i am go school tomorrow
```

Output:

```
I am going to school tomorrow.
```

2: Structured Note Formatting

```
math exam monday
study algebra
buy pen
```

Output:

- Formatted paragraph
- Bullet list tasks extracted

MILESTONE 2: Core Functionalities Development (Streamlit)

File Explorer Structure

```
scribble_to_digital/
│
├── app.py                  # Main Streamlit application
├── .env                    # API key storage
├── requirements.txt        # Python dependencies
│
├── utils/
│   ├── ocr_processor.py    # Image preprocessing & OCR
│   ├── ai_processor.py     # Gemini integration
│   └── file_export.py      # Export functions (PDF / TXT)
│
├── assets/
│   └── sample_images/
│
└── outputs/                # Generated files
```

Activity 2.1: Streamlit Initialization and Import Libraries

In app.py:

```
import streamlit as st
import google.generativeai as genai
import os
from dotenv import load_dotenv
from PIL import Image
import pytesseract
import cv2
import numpy as np
```

Configure Gemini Model

```
load_dotenv()
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

model = genai.GenerativeModel("models/gemini-2.5-flash")
```

Activity 2.2: OCR Processing Module

```
import cv2
import pytesseract
import numpy as np

def extract_text_from_image(uploaded_image):
    image = np.array(uploaded_image)
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    gray = cv2.threshold(gray, 150, 255, cv2.THRESH_BINARY)[1]

    text = pytesseract.image_to_string(gray)
    return text
```

Activity 2.3: Gemini AI Processing Module

```python
import google.generativeai as genai

model = genai.GenerativeModel("models/gemini-2.5-flash")

def clean_and_structure_text(raw_text):
    prompt = f"""
    You are an AI note cleaner.
    1. Correct spelling and grammar.
    2. Reconstruct incomplete words.
    3. Format text properly.
    4. Extract action items as bullet points.

    Raw Text:
    {raw_text}
    """

    response = model.generate_content(prompt)
    return response.text
```

Inside app.py

```python
st.title("✍️ Scribble to Digital")

uploaded_file = st.file_uploader(
    "Upload Handwritten Notes Image",
    type=["jpg", "png", "jpeg"]
)

if uploaded_file is not None:
    image = Image.open(uploaded_file)
    st.image(image, caption="Uploaded Image", use_column_width=True)

    if st.button("Convert to Digital"):

        raw_text = extract_text_from_image(image)

        st.subheader("📜 Extracted Raw Text")
        st.text_area("OCR Output", raw_text, height=150)

        clean_text = clean_and_structure_text(raw_text)

        st.subheader("✨ Cleaned & Structured Output")
        st.write(clean_text)
```

Save as TXT

```python
def save_as_txt(content, filename):
    with open(f"outputs/{filename}.txt", "w", encoding="utf-8") as f:
        f.write(content)
```

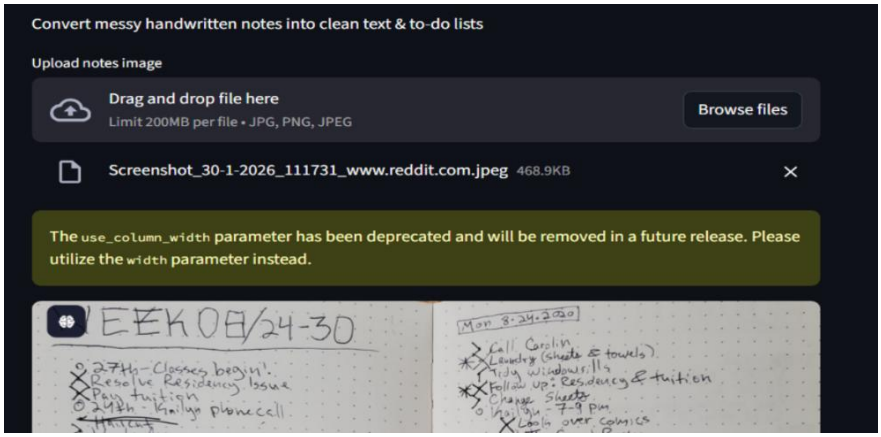
In app.py:

```python
if st.button("Download as TXT"):
    save_as_txt(clean_text, "digital_notes")
    st.success("File saved in outputs folder.")
```

Main Application Structure

```
scribble_to_digital/
|
├── app.py                # Main Flask server
├── routes/
|   └── process.py        # OCR + AI logic routes
├── services/
|   ├── ocr_service.py    # Image enhancement + OCR
|   ├── ai_service.py     # Gemini processing
|   └── export_service.py # PDF/Word export
├── utils/
|   └── validators.py     # File validation
└── uploads/
```

OUTPUT :



CONCLUSION

The Scribble to Digital system provides an efficient solution for converting handwritten notes into organized digital text. By integrating image enhancement, OCR technology, and Generative AI, the system improves text readability, corrects grammar, and extracts important tasks automatically.

This project helps students and professionals save time, improve productivity, and maintain well-organized digital notes. In the future, the system can be improved with additional features such as multi-language support, mobile integration, and cloud storage.

References

- **Visual Studio Code (VS Code)** – Source code editor used for developing and running the Generative AI application.
- **Python Programming Language** – Used as the primary programming language for implementing backend logic and AI model integration.
- **Google Cloud Platform (GCP)** – Used for deploying AI services and managing cloud-based infrastructure for the application.
- **Vertex AI** – Used for building, training, and deploying machine learning and generative AI models.
- **Vector Search and Embeddings** – Used for converting text data into vector representations and performing similarity searches for intelligent information retrieval.
- **Google Gemini AI** – Used for advanced generative AI capabilities such as natural language understanding, reasoning, and response generation.
- **Streamlit Framework** – Used to develop an interactive web-based user interface for the AI application.